

AI MODELS COMPARATOR

Software Requirements Specification (SRS)

Software Engineering – A4 – 2026-2027

Vivien Gruel
Noé Wales
Valentin Gempp
Maxence Tessier

Contents

1	Introduction	2
1.1	Purpose	2
1.2	Scope	2
1.3	Definitions, Acronyms, and Abbreviations	2
1.4	References	2
1.5	Document Overview	2
2	General Description	2
2.1	Product Perspective	2
2.2	Product Features	3
2.3	User Characteristics	3
2.4	General Constraints	3
2.5	Assumptions and Dependencies	3
3	Specific Requirements	3
3.1	Functional Requirements	3
3.1.1	Model Catalogue	3
3.1.2	Benchmark Ranking	4
3.1.3	Community Ranking	4
3.1.4	Ranking Comparison	4
3.1.5	Comments	4
3.1.6	AI News	5
3.1.7	User Accounts	5
3.1.8	Administration	5
3.2	Non-Functional Requirements	5
3.2.1	Performance	5
3.2.2	Security	5
3.2.3	Usability	6
3.2.4	Reliability	6
3.2.5	Maintainability	6
4	Use Case Scenarios	6
4.1	Use Case 1: Voting for a Model	6
4.2	Use Case 2: Commenting on a Model	7
5	Modeling Diagrams	8
5.1	Class Diagram	8
5.2	Sequence Diagram	9
5.3	Use Case Diagram	10
6	External Interfaces	12
6.1	User Interface	12
6.2	Hardware Interfaces	12
6.3	Software Interfaces	13
6.4	Communication Interfaces	13
7	Other Requirements	13
7.1	Security and Privacy	13
7.2	Compatibility	13
7.3	Maintainability	13

8	Appendices	13
9	Index	13

1 Introduction

1.1 Purpose

This document describes the functional and non-functional requirements of the AI Models Comparator. It explains **what** the system must do. It is written for the developers, the testers and the TD instructor, so that everybody has the same understanding of the project.

1.2 Scope

The AI Models Comparator is a web application that ranks AI models in two ways: with benchmark scores collected from external websites, and with the up and down votes of the users. Each model has its own page with its scores, its rankings and a comment feed. The website also shows news about new AI models. Administrators can moderate comments and users.

1.3 Definitions, Acronyms, and Abbreviations

- **AI model / LLM:** a Large Language Model, for example GPT-5 or Claude.
- **Lab:** the company that created a model (for example OpenAI, Anthropic, Meta).
- **Benchmark:** a standard test used to measure the performance of a model (for example GPQA or MMLU-Pro).
- **Community score:** percentage of up votes that a model received.
- **Benchmark score:** average of the benchmark results of a model.
- **Slug:** a short text used in the URL of a model (for example `gpt-4o-2024-08-06`).
- **SPA:** Single Page Application.
- **API:** Application Programming Interface.
- **GDPR:** General Data Protection Regulation.

1.4 References

- Business Requirements Document (BRD) of the AI Models Comparator.
- IEEE 830 / ISO/IEC/IEEE 29148 for requirements specification.
- SHMS SRS template given in the course.

1.5 Document Overview

Section 2 gives a general description of the product. Section 3 lists the functional and non-functional requirements. Section 4 describes the main use cases. Section 5 contains the UML diagrams. Section 6 describes the external interfaces and Section 7 the other requirements.

2 General Description

2.1 Product Perspective

The AI Models Comparator is a new, independent web application. It has a React frontend and a Node.js/Express backend with a PostgreSQL database. It depends on external websites for the benchmark scores and the AI news. These data are imported automatically by the backend and stored in the database, so the users never call the external websites directly.

2.2 Product Features

- **Model catalogue:** all models grouped by lab.
- **Model page:** details, benchmark scores, rankings, votes and comments of one model.
- **Benchmark ranking:** ranking based on the benchmark scores.
- **Community ranking:** ranking based on the up and down votes.
- **Comments:** one comment feed for each model.
- **AI news:** list of news about AI models.
- **Accounts:** sign up, log in, log out, delete account.
- **Administration:** moderation of comments and users, status of the imports.

2.3 User Characteristics

- **Visitor:** any person on the website without an account. Can read everything but cannot vote or comment.
- **Registered user:** a visitor with an account. Can vote, comment, and delete their own comments and account. Usually developers, students or AI fans.
- **Administrator:** a registered user with the ADMIN role. Can moderate all comments and users.

2.4 General Constraints

- The frontend must be developed with React and the backend with Node.js (course requirement).
- The system must respect the GDPR.
- The system must respect the terms of use of the external data sources.
- The project must be finished during the sprints of the course, by a team of four students.

2.5 Assumptions and Dependencies

- At least one external website gives benchmark scores with an API or a downloadable file.
- At least one free RSS feed or API gives news about AI models.
- Users have a recent web browser and an internet connection.

3 Specific Requirements

3.1 Functional Requirements

3.1.1 Model Catalogue

- **FR-1.1:** The system must display all the labs in alphabetical order, each with its logo and name.
- **FR-1.2:** Under each lab, the system must display the names of its models in alphabetical order.

- **FR-1.3:** When the user clicks on a model, the system must open the model page at `/models/{slug}`.
- **FR-1.4:** The model page must show the name, the lab, the release date, the two ranks, the table of benchmark scores (benchmark, score, source), the community score with the number of votes, and the comment feed.
- **FR-1.5:** If the slug does not exist, the system must display a “Model not found” page (404).

3.1.2 Benchmark Ranking

- **FR-2.1:** The system must import benchmark scores automatically every 24 hours.
- **FR-2.2:** The system must only keep benchmarks expressed as a percentage (0 to 100).
- **FR-2.3:** The benchmark score of a model is the average of all its benchmark results.
- **FR-2.4:** A model is in the benchmark ranking only if it has at least 3 benchmark results. Otherwise, it is shown in a “Not enough benchmarks” list.
- **FR-2.5:** Each score must show its source and the date when it was imported.

3.1.3 Community Ranking

- **FR-3.1:** A logged-in user can vote up or down for a model. There is only one vote per user and per model.
- **FR-3.2:** If the user clicks on the selected arrow again, the vote is removed. If the user clicks on the other arrow, the vote is changed.
- **FR-3.3:** The community score of a model is: $up\ votes / (up\ votes + down\ votes) \times 100$.
- **FR-3.4:** A model is in the community ranking only if it has at least 10 votes. Otherwise, it is shown in a “Not enough votes” list.
- **FR-3.5:** A visitor who clicks on an arrow sees the message “Log in to vote”.

3.1.4 Ranking Comparison

- **FR-4.1:** The ranking page must show one ranking at a time, with a button to switch between “Community” and “Benchmarks”.
- **FR-4.2:** Each line must show the rank, the model, the lab, the score, and the rank of the model in the other ranking.
- **FR-4.3:** In case of equal scores, the model with more votes (or more benchmarks) is ranked first.

3.1.5 Comments

- **FR-5.1:** Each model has one comment feed, without replies. The newest comment is on top.
- **FR-5.2:** A logged-in user can post as many comments as they want. A comment has between 1 and 1000 characters.
- **FR-5.3:** Each comment shows the username of the author, the date and the text.

- **FR-5.4:** A user can delete their own comments.
- **FR-5.5:** A visitor sees the comments but the form is replaced by “Log in to comment”.

3.1.6 AI News

- **FR-6.1:** The system must import news automatically every 6 hours.
- **FR-6.2:** The news page must show the news from the newest to the oldest, with the title, the source and the date.
- **FR-6.3:** When the user clicks on a news, the original article opens in a new tab.

3.1.7 User Accounts

- **FR-7.1:** A visitor can create an account with an email, a username and a password. The email and the username must be unique.
- **FR-7.2:** The password must have at least 8 characters.
- **FR-7.3:** The visitor must accept the privacy policy to create an account.
- **FR-7.4:** A user can log in with the email and the password, and log out.
- **FR-7.5:** A blocked user cannot log in and sees the message “Your account is blocked”.
- **FR-7.6:** A user can delete their account after a confirmation. All their votes and comments are deleted.

3.1.8 Administration

- **FR-8.1:** Only users with the ADMIN role can open the administration page.
- **FR-8.2:** An administrator can delete any comment.
- **FR-8.3:** An administrator can block or unblock a user, from the administration page or from a comment.
- **FR-8.4:** An administrator can delete a user account.
- **FR-8.5:** The administration page shows, for each source, the date of the last successful import and the last error.

3.2 Non-Functional Requirements

3.2.1 Performance

- **NFR-1:** Each page must load in less than 2 seconds with up to 100 users at the same time.
- **NFR-2:** A vote must be saved and the new score displayed in less than 1 second.

3.2.2 Security

- **NFR-3:** Passwords must be hashed with bcrypt. They are never stored in clear text.
- **NFR-4:** All communications must use HTTPS.
- **NFR-5:** The system must use role-based access control (visitor, user, admin) on every API route.
- **NFR-6:** All user inputs must be validated on the server side.

3.2.3 Usability

- **NFR-7:** Every page must work on a phone (360 px wide screen) and on a computer.
- **NFR-8:** Every page must handle the loading, empty and error states.

3.2.4 Reliability

- **NFR-9:** If an import fails, the system must keep the last valid data and save the error.

3.2.5 Maintainability

- **NFR-10:** The code must be documented and follow the same style (ESLint, Prettier).
- **NFR-11:** Unit tests must cover the score calculations and the vote rules.
- **NFR-12:** A new source must be added by writing one new importer, without changing the rest of the code.

4 Use Case Scenarios

4.1 Use Case 1: Voting for a Model

Actors: Registered User, System

Description: the user gives an opinion on a model by clicking on the up or down arrow on the model page.

Precondition: the user is logged in and the account is active.

Steps:

1. The user opens the page of a model.
2. The system shows the model with its current community score and the vote of the user (if any).
3. The user clicks on the up or down arrow.
4. The system adds, removes or changes the vote.
5. The system computes the new community score.
6. The system shows the new score and the selected arrow.

Alternative flow: at step 3, if the user is not logged in, the system shows “Log in to vote”.

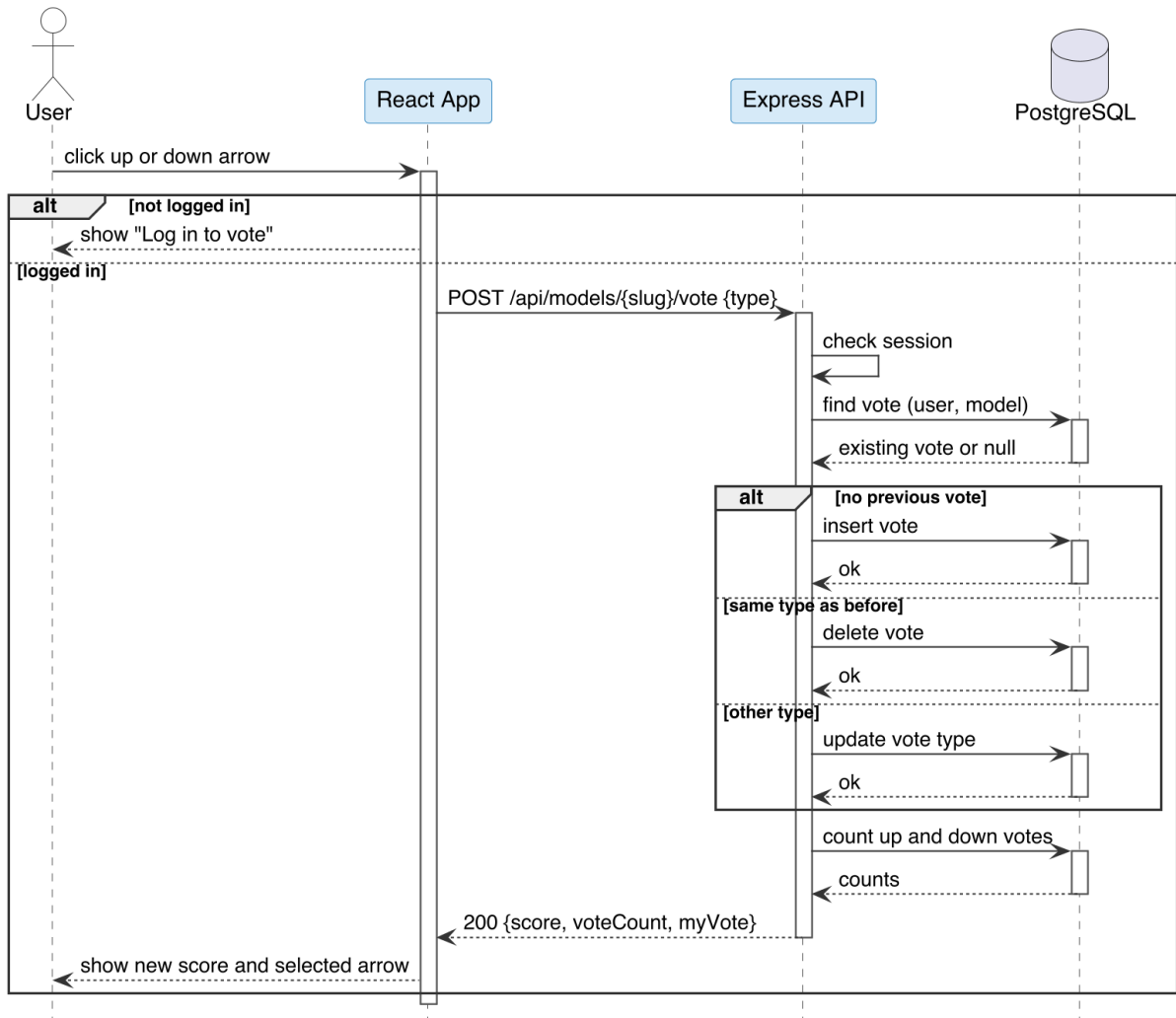


Figure 1: Sequence diagram of UC1 – Voting for a model

4.2 Use Case 2: Commenting on a Model

Actors: Registered User, System

Description: the user writes a comment in the feed of a model to discuss its performance.

Precondition: the user is logged in and the account is active.

Steps:

1. The user opens the page of a model.
2. The user writes a comment and clicks on “Send”.
3. The system checks that the comment is not empty and has less than 1000 characters.
4. The system saves the comment.
5. The system shows the comment at the top of the feed.

Alternative flows: if the user is not logged in, the system asks the user to log in. If the comment is empty or too long, the system shows an error and the user can correct it.

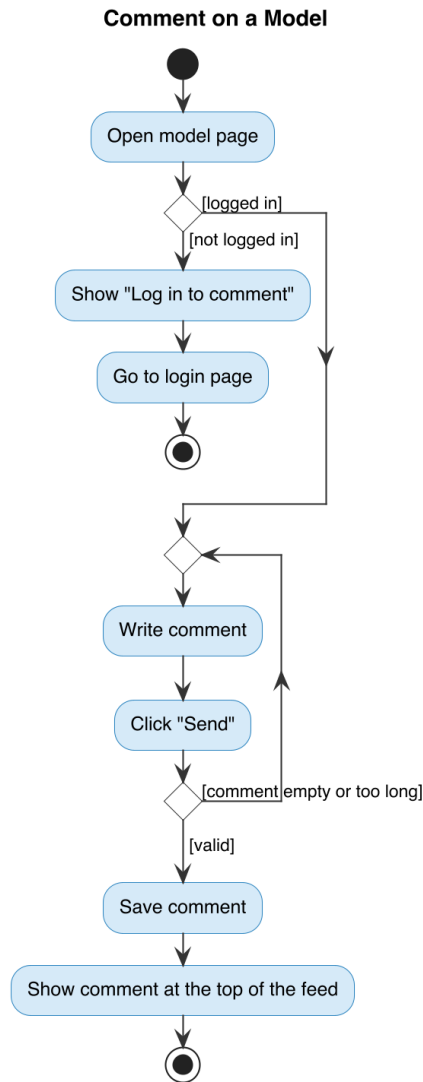


Figure 2: Activity diagram of UC2 – Commenting on a model

5 Modeling Diagrams

5.1 Class Diagram

The class diagram shows the main classes of the system, organized in five packages. **Vote** and **Score** are association classes: there is exactly one vote for each pair (user, model) and one score for each pair (model, benchmark). **Comment** is a normal class because a user can write many comments on the same model. The composition between **User** and **Comment** means that the comments are deleted with the account.

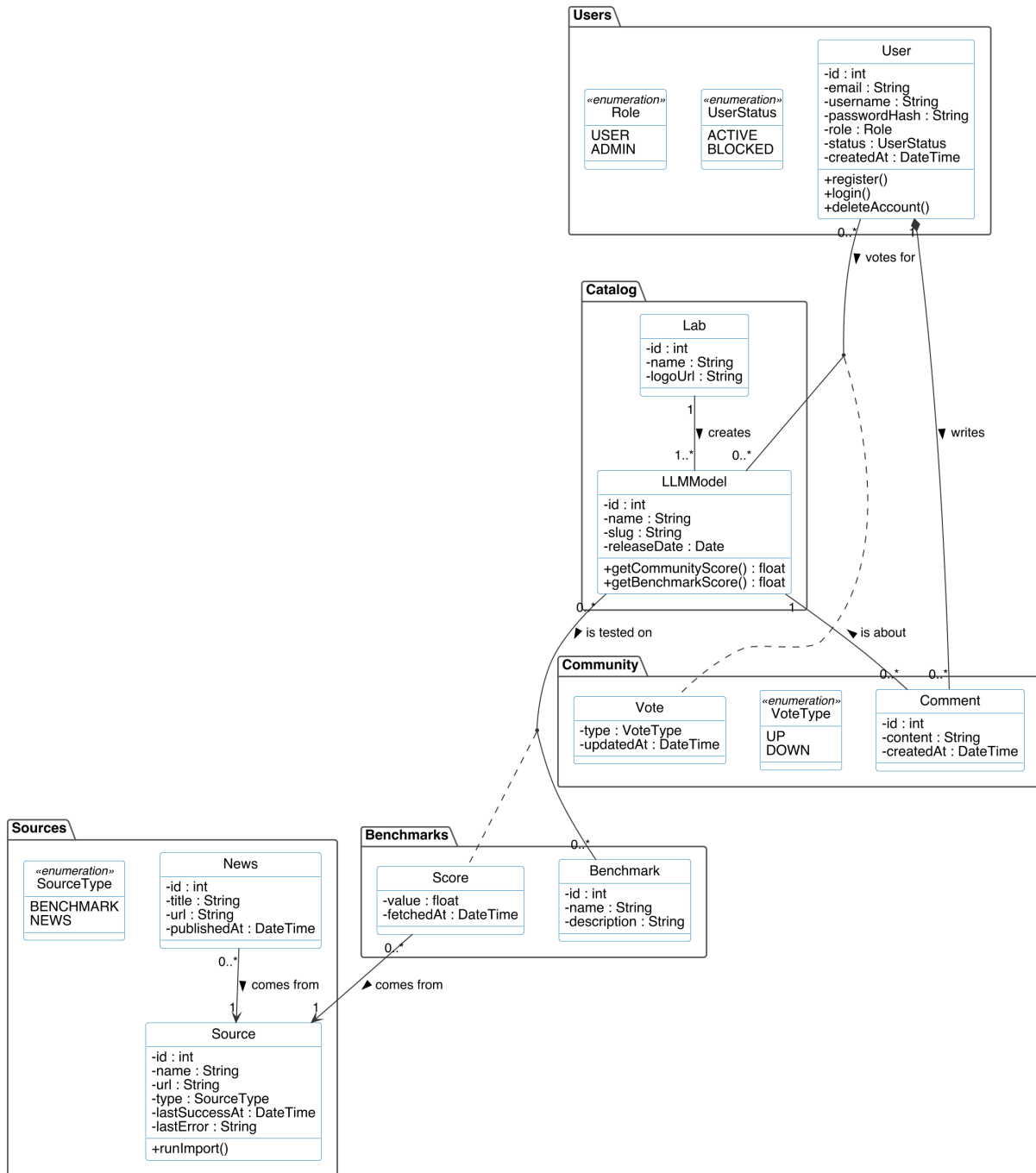


Figure 3: Class diagram with packages

5.2 Sequence Diagram

This diagram shows how the model page is loaded, including the case where the model does not exist.

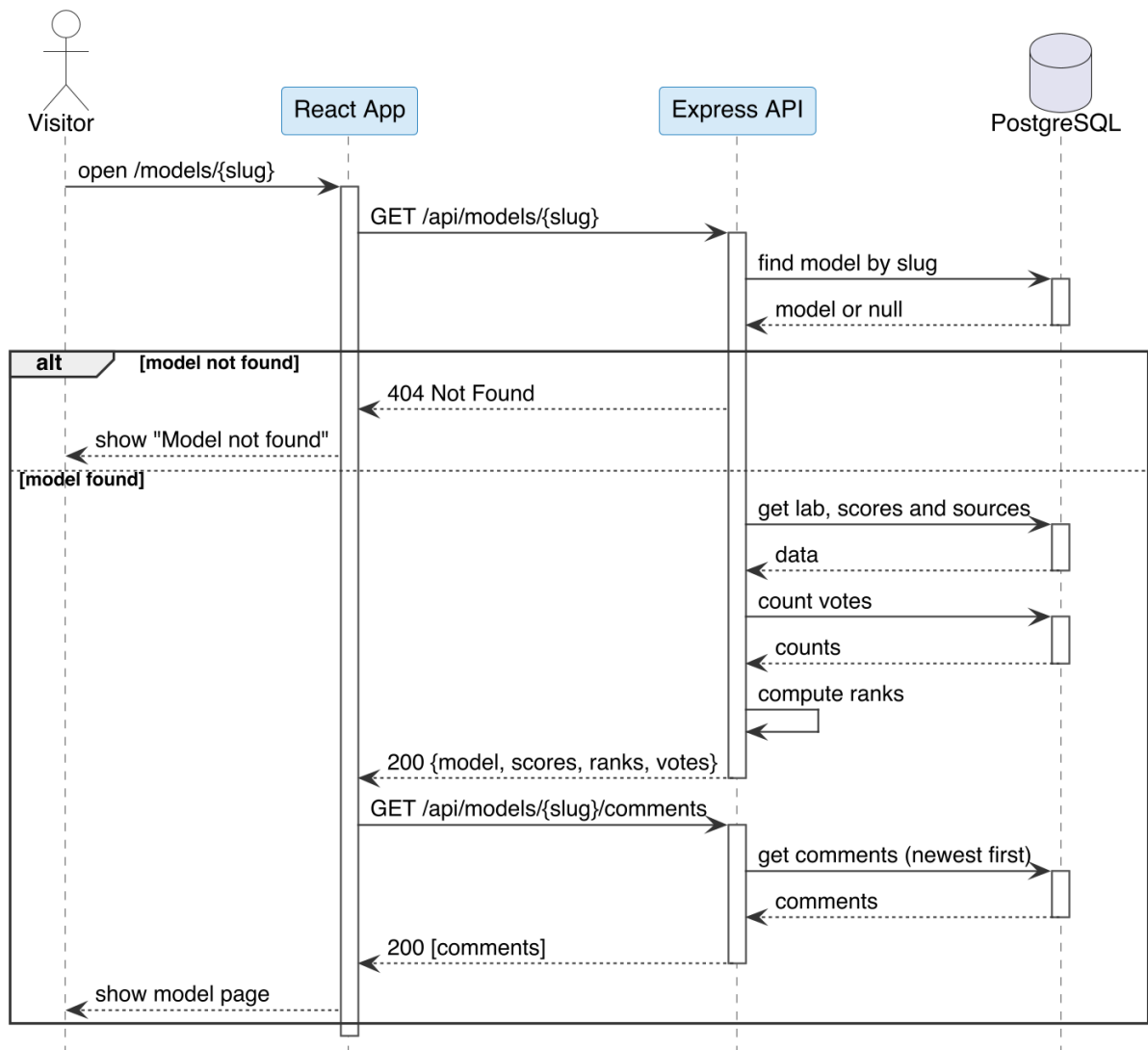


Figure 4: Sequence diagram – Opening a model page

5.3 Use Case Diagram

The Registered User inherits the use cases of the Visitor, and the Administrator inherits the use cases of the Registered User. Voting and commenting include logging in. “Block author” extends “Delete any comment”.

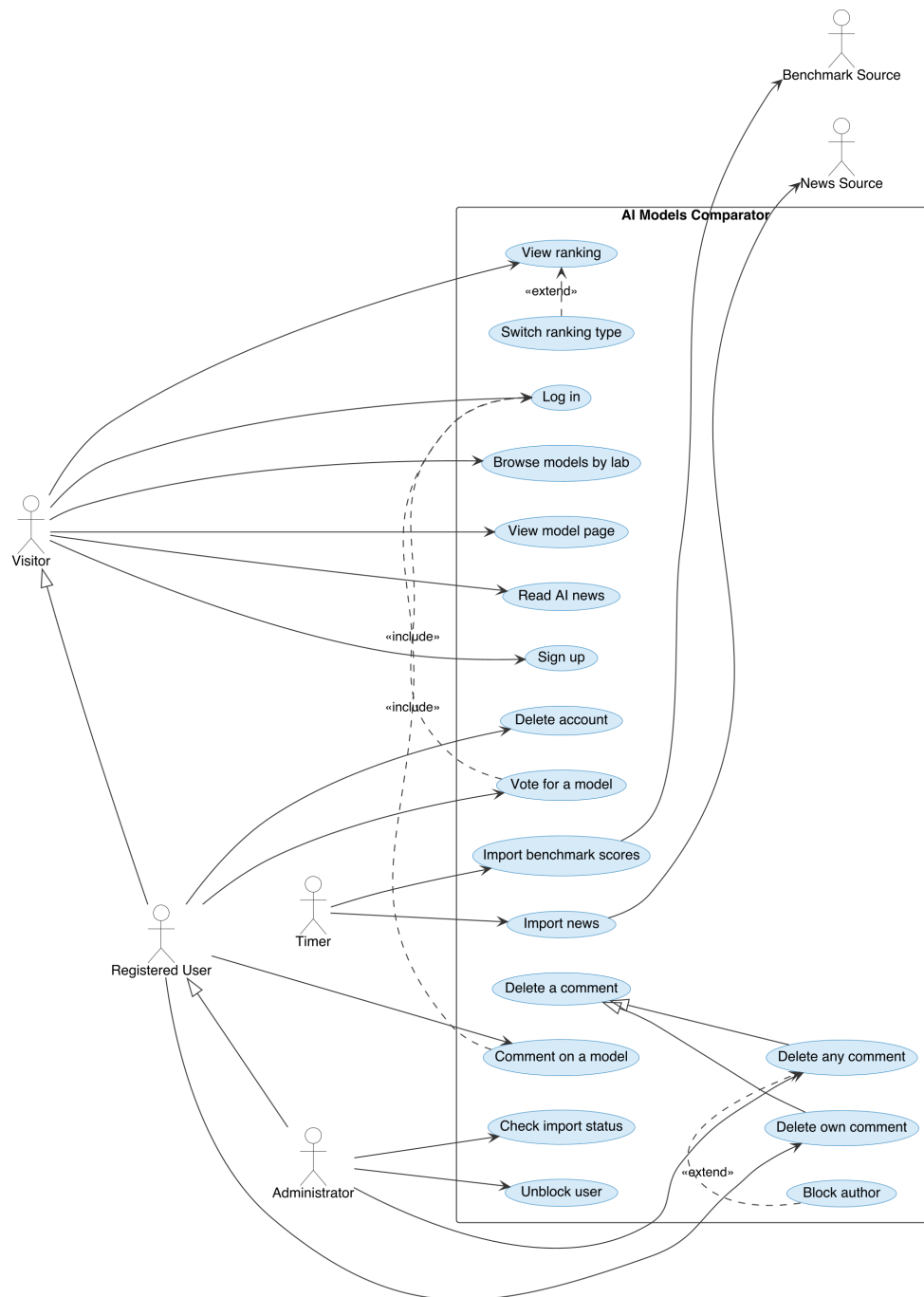


Figure 5: Use case diagram

6 External Interfaces

6.1 User Interface

Route	Access	Content
/	Public	Title and three buttons: Models, Ranking, News
/login	Public	Log in and Sign up tabs
/models	Public	Labs with their logo, and their models (alphabetical order)
/models/{slug}	Public	Model details, benchmarks, votes, comments
/ranking	Public	Community or benchmark ranking, with the other rank on each line
/news	Public	AI news, newest first
/account	User	Username, email, “Delete my account” button
/admin	Admin	Users, recent comments, import status

Wireframes (the names and numbers are only examples):

Figure 6: Login / Sign up

Lab	Model
Anthropic	Claude Opus
Google	Gemini Pro
Meta	Llama 4
Anthropic	Claude Sonnet
Google	Gemini Flash
Meta	Llama 3.3
Mistral	Mistral Large
OpenAI	GPT-5
xAI	Grok 4
Mistral	Codestral
OpenAI	GPT-4o
xAI	Grok 3

Figure 7: Models page

Benchmark	Score	Source
GPQA	85.7 %	Source A
MMLU-Pro	87.1 %	Source A
LiveCodeBench	74.9 %	Source B

Figure 8: Model page

#	Model	Lab	Score	Benchmark rank
1	Claude Opus	Anthropic	91 % up	2nd
2	Gemini Pro	Google	88 % up	3rd
3	GPT-5	OpenAI	82 % up	1st
4	Llama 4	Meta	75 % up	6th

Figure 9: Ranking page

6.2 Hardware Interfaces

The system does not use any special hardware. It works on any computer or phone with a web browser.

6.3 Software Interfaces

- **Benchmark source API:** the backend calls a public API (REST, JSON) to get the benchmark scores.
- **AI news feed:** the backend reads a public RSS feed (XML) to get the news.
- **PostgreSQL:** the backend stores all the data in a PostgreSQL database.

6.4 Communication Interfaces

- The frontend and the backend communicate with a REST API over HTTPS, using JSON.
- The user session is stored in a secure cookie (HttpOnly, Secure).

7 Other Requirements

7.1 Security and Privacy

- We only store the email, the username and the hashed password of the users (GDPR data minimisation).
- The privacy policy is shown at sign up and must be accepted.
- When an account is deleted, all its personal data, votes and comments are deleted.
- The database is saved every day.

7.2 Compatibility

- The website must work on the last two versions of Chrome, Firefox, Safari and Edge.

7.3 Maintainability

- The code is split into modules (one service for each feature) so that a feature can change without breaking the others.

8 Appendices

- **Appendix A:** wireframes of the main pages (Section 6.1).
- **Appendix B:** REST API documentation (see the SDD, Section 5).

9 Index

- **Benchmark ranking:** 3.1.2
- **Comments:** 3.1.5, 4.2
- **Community ranking:** 3.1.3, 4.1
- **GDPR:** 2.4, 7.1
- **Vote:** 3.1.3, 4.1, 5.1